

贵州师范大学

作业（论文）题目：基于 Spring Boot 与 Next.js 的智能 Agent 平台设计与实现

课程名称：软件综合设计与实践

任课教师姓名：李伟岸

小组成员 1：李睿

学 号：252200232274

专 业：网络与信息安全

小组成员 2：曾钰鹏

学 号：252200232260

专 业：大数据技术与工程

小组成员 3：黄皓远

学 号：252200232249

专 业：计算机技术

年 级：2025 级

学院（部、所）：大数据与计算机科学学院

目 录

1 项目概述 7

 1.1 项目背景 7

 1.2 项目目标 8

 1.3 项目规模与代码结构 8

 1.4 技术路线 9

2 需求分析 10

 2.1 功能需求 10

 2.2 非功能需求 11

 2.3 角色与使用场景 11

3 总体架构设计 12

 3.1 系统总体架构 12

 3.2 后端分层架构 12

 3.3 前端架构设计 13

 3.4 部署架构设计 14

4 核心模块设计与实现 14

 4.1 用户认证与权限模块 14

 4.2 Agent 创建与会话模块 15

 4.3 RAG 知识库模块 15

 4.4 工具市场与工具安装模块 16

 4.5 管理后台模块 16

 4.6 支付、用量与 API Key 模块 16

- 4.7 容器管理与 Web Terminal 模块 17
- 5 数据库与数据流设计 17
 - 5.1 核心数据对象 17
 - 5.2 数据关系设计 18
 - 5.3 Agent 会话数据流 18
 - 5.4 RAG 文件处理数据流..... 19
 - 5.5 统一响应与前后端接口 19
- 6 系统实现与运行部署 19
 - 6.1 后端实现要点 19
 - 6.2 前端实现要点 20
 - 6.3 部署与启动方式 20
 - 6.4 配置与安全注意事项 21
 - 6.5 线上部署与演示访问 21
- 7 测试与验证 21
 - 7.1 测试思路 21
 - 7.2 功能测试用例 22
 - 7.3 接口与部署测试 22
 - 7.4 测试结论 23
- 8 项目难点与解决方案 23
 - 8.1 多业务域带来的复杂度 23
 - 8.2 RAG 处理链路较长..... 23
 - 8.3 Agent 会话需要兼顾实时性和持久化 24

8.4 部署依赖较多	24
8.5 安全与配置管理	24
9 总结与展望	25
9.1 工作总结	25
9.2 不足之处	25
9.3 后续展望	25
附录 A 主要代码说明	26
附录 B 系统运行效果截图	26

成员分工说明

本课程作业由三名成员协作完成，具体分工如下表所示。三位成员在需求讨论、功能联调和报告梳理过程中均有参与。

成员	主要完成内容
成员一（姓名： <u>李睿</u> ）	主要负责后端核心功能实现，包括 Agent 会话流程、RAG 知识库处理、数据库设计、接口开发与云服务器部署。
成员二（姓名： <u>曾钰鹏</u> ）	主要负责前端页面实现与交互联调，包括工作台、知识库、工具市场、管理后台等页面，以及前后端接口联调。
成员二（姓名： <u>黄皓远</u> ）	成员三：主要负责系统测试、功能验证、运行截图整理、课程报告与 PPT 内容梳理、格式排版和演示材料完善。

1 项目概述

1.1 项目背景

近年来，大语言模型在自然语言理解、文本生成、代码辅助和知识问答方面表现出较强能力，但在真实应用中仍存在三类常见问题：第一，模型无法直接访问业务系统和外部工具；第二，模型自身参数中的知识无法覆盖企业或个人的私有文档；第三，模型调用过程缺少用户、权限、任务和运维层面的工程支撑。因此，面向实际使用的 AI 应用需要从单一聊天界面演进为可配置、可扩展、可管理的智能体平台。

AgentX 项目正是在这一背景下设计的。它将大模型能力与工具调用、知识库检索、会话管理、用户系统、管理后台和容器化部署结合起来，使用户可以创建自己的 Agent，配置模型和工具，上传知识库文档，并通过 Web 页面或 Widget 方式进行智能问答。相较于普通聊天应用，AgentX 更强调平台能力和工程可维护性。

1.2 项目目标

本项目的目标不是实现一个单点 Demo，而是构建一个具有完整业务链路的智能体平台。具体而言，系统需要支持普通用户创建和使用 Agent，支持管理员对模型、工具、知识库和用户进行管理，支持知识库文件的上传、处理和检索，支持工具市场的安装和发布，同时能够通过 Docker Compose 进行本地或服务器环境部署。

本项目希望体现三个层次的工作量：一是前后端完整业务功能的实现；二是后端按照领域和应用服务进行分层设计；三是对 AI 应用特有问题，如 RAG 检索、流式会话、工具扩展和异步任务处理进行工程化组织。这样既避免只停留在界面展示，也避免过度偏向理论而脱离系统实现。

1.3 项目规模与代码结构

当前工作区由三个主要部分组成：AgentX 后端、agentx-frontend-plus 前端和 deploy 部署目录。根据源码文件统计，项目中 Java 文件约 754 个，TSX 文件约 251 个，TypeScript 文件约 93 个，说明系统已经形成了较完整的业务模块和工程结构。后端负责业务逻辑、数据库访问、第三方服务集成和基础设施能力；前端负责用户交互、页面路由和接口调用；部署目录负责容器镜像构建和服务编排。

表 1-1 项目主要目录与作用

目录或文件	作用说明
AgentX/pom.xml	后端 Maven 配置，声明 Spring Boot、MyBatis-Plus、pgvector、RabbitMQ、支付、文件解析等依赖。
AgentX/src/main/java/org/xhy	后端核心源码，按照 application、domain、infrastructure、interfaces 分层组织。
AgentX/src/main/resources/application.yml	后端运行配置，包括端口、数据库、RabbitMQ、文件存储、向量检索、支付和容器管理等。
agentx-frontend-plus/package.json	前端依赖与脚本配置，包含 Next.js、React、Tailwind、Radix UI、xterm 等依赖。
agentx-frontend-plus/app	Next.js App Router 页面结构，包含认证、主应用、知识库、工具、管理后台等页面。

agentx-frontend-plus/components	通用组件和业务组件，如聊天面板、知识库对话框、工具安装、支付组件等。
deploy/docker-compose.yml	统一编排 PostgreSQL/pgvector、RabbitMQ、后端、前端和可选 Adminer。
deploy/start.bat / start.sh	本地启动脚本，便于快速构建并运行完整系统。

1.4 技术路线

项目采用较典型的现代 Web 全栈技术路线。后端使用 Spring Boot 提供 REST API、WebSocket/SSE、定时任务和业务服务；数据库使用 PostgreSQL 保存结构化业务数据，并通过 pgvector 支持向量检索；RabbitMQ 用于异步任务和消息事件；前端使用 Next.js 和 React 构建复杂页面，配合 Tailwind CSS 和 shadcn/ui 保证界面一致性；Docker Compose 将运行依赖统一编排，使系统具备较好的可部署性。

表 1-2 项目技术选型

技术类别	选型	在项目中的作用
后端框架	Spring Boot 3.2.3 / Java 17	构建业务服务、接口、配置体系和运行容器。
数据访问	MyBatis-Plus	简化数据库实体、仓储和分页查询实现。
前端框架	Next.js 15 / React 19	构建页面路由、管理后台、工作台和 Widget 页面。
样式组件	Tailwind CSS / shadcn/ui / Radix UI	形成统一、可复用的界面组件体系。
数据库	PostgreSQL + pgvector	保存业务数据，并支持知识片段向量检索。
消息队列	RabbitMQ	支撑文件处理、同步任务等异步流程。
部署	Docker Compose	统一启动数据库、消息队列、前后端服务。
智能能力	多模型供应商、RAG、工具市场	将大模型、私有知识和外部工具纳入统一平台。

2 需求分析

2.1 功能需求

从系统使用者角度看，AgentX 面向三类主体：普通用户、管理员和外部调用方。普通用户关注 Agent 创建、对话、知识库和工具使用；管理员关注平台资源、审核、用户和订单；外部调用方关注 API Key、Widget 和公开接口。因此需求分析需要覆盖平台内部管理和对外服务两个方向。

表 2-1 系统功能需求

需求类别	具体需求	说明
用户与认证	注册、登录、重置密码、用户设置、管理员权限	保证系统使用入口和权限边界。
Agent 管理	创建 Agent、编辑配置、版本发布、加入工作区	使用户能够配置专属智能体。
会话交互	创建会话、发送消息、流式响应、任务中断、历史记录	支撑核心智能问答体验。
模型配置	模型供应商、模型类型、默认模型、用户模型配置	适配不同大模型服务。
工具市场	工具上传、安装、卸载、版本管理、管理员审核	扩展 Agent 可调用能力。
RAG 知识库	数据集创建、文件上传、解析、切分、向量化、检索和问答	解决模型私有知识不足的问题。
记忆管理	记忆条目创建、查询、删除和向量存储	提升多轮对话和个性化能力。
支付与用量	商品、订单、充值、支付状态轮询、用量记录	支持平台商业化或额度管理。
管理后台	用户、模型、工具、知识库、订单、容器模板等管理	支撑平台运维和内容审核。
部署运维	健康检查、日志、Docker 编排、本地启动脚本	降低部署和调试复杂度。

2.2 非功能需求

除了功能完整性，本项目还需要满足一定的非功能需求。智能体平台通常涉及较多外部依赖，包括模型服务、对象存储、数据库、消息队列和容器运行环境。如果缺少合理的配置和分层，后续维护成本会很高。因此，项目在设计上强调可扩展、可部署、可维护和基本安全性。

表 2-2 非功能需求

非功能需求	设计考虑
可维护性	后端按接口、应用、领域和基础设施分层，前端按页面、组件、服务和类型文件组织。
可扩展性	模型供应商、工具、RAG 数据集、支付渠道等均以独立模块组织，便于后续扩展。
可部署性	通过 Docker Compose 统一编排数据库、消息队列、前后端服务，降低环境差异。
安全性	采用登录认证、管理员权限、API Key 等机制，同时敏感配置应进一步环境变量化。
可观测性	提供健康检查接口、日志配置和任务状态查询，便于定位运行问题。
用户体验	前端提供对话、上传、审核、管理等完整界面，减少用户直接操作接口的成本。

2.3 角色与使用场景

普通用户的典型流程是注册登录后创建 Agent，选择模型，安装需要的工具，创建知识库并上传文档，然后在工作台中进行问答。管理员的典型流程是登录管理后台，维护模型供应商、审核用户上传的工具或知识库、查看订单和用户状态。外部调用方则可以通过 API Key 或 Widget 将 Agent 能力嵌入其他系统中。

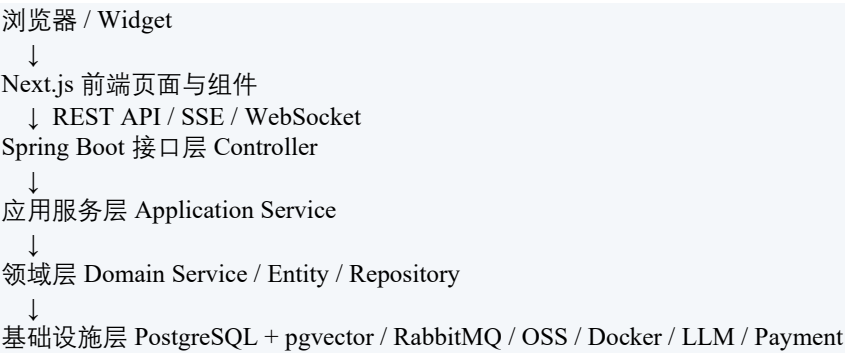
这种角色划分使系统不仅是一个面向个人的聊天工具，更接近一个可运营的平台。课程报告中重点分析普通用户和管理员两个主要角色，因为它们最能体现项目的业务完整度和工程实现工作量。

3 总体架构设计

3.1 系统总体架构

AgentX 采用前后端分离和后端分层相结合的总体架构。前端通过 REST API、SSE 或 WebSocket 与后端交互，后端在应用服务层组织具体业务流程，在领域层沉淀业务规则，在基础设施层适配数据库、消息队列、文件存储、Docker、LLM 服务和支付服务。整体结构既符合 Web 应用开发习惯，也便于智能体业务模块独立演进。

图 3-1 系统总体架构示意



3.2 后端分层架构

后端源码位于 AgentX/src/main/java/org/xhy 下，主要分为 application、domain、infrastructure 和 interfaces 四层。interfaces 层负责暴露 Controller 和接收请求；application 层负责编排用例流程，例如创建 Agent、发起会话、处理知识库文件；domain 层负责业务对象、领域服务和仓储接口；infrastructure 层负责数据库、Docker、邮件、支付、向量检索、消息队列等外部资源适配。

表 3-1 后端分层职责

后端层次	主要职责	示例
interfaces	接收 HTTP 请求、参数校验、统一响应	PortalAgentController、AdminToolController、RagSearchController
application	组织业务用例，协调多个领域服务和基础设施能力	Agent 应用服务、RAG 管理服务、订单应用服务

domain	沉淀核心业务规则、实体、领域服务和仓储接口	UserDomainService、ContainerDomainService、OrderDomainService
infrastructure	实现外部系统适配和通用技术能力	DockerService、LLMProviderService、RabbitMQ、支付 Provider

这种分层方式的优点是可以减少 **Controller** 中的业务堆积，也可以避免基础设施细节直接侵入业务规则。例如，RAG 文件处理需要涉及文件上传、文档解析、切片、向量化和消息事件，如果全部写在接口层会导致代码难以维护；通过应用层编排和基础设施层适配，可以把复杂流程拆分到更清晰的位置。

3.3 前端架构设计

前端位于 `agentx-frontend-plus` 目录，采用 `Next.js App Router`。页面主要分为认证页面、主应用页面、**Widget** 页面和管理后台页面。业务接口封装在 `lib` 目录中，类型定义集中在 `types` 目录，常用 UI 组件放在 `components/ui`，业务组件则按照 **Agent**、**RAG**、工具、支付、管理后台等功能划分。

这种组织方式有利于前端在功能变多时保持可读性。比如知识库相关页面并不直接拼接所有请求，而是通过 `rag-dataset-service`、`rag-file-service`、`rag-chat-service` 等服务文件访问后端；通用 `Button`、`Dialog`、`Tabs`、`Table` 等组件由 `shadcn/ui` 风格组件统一提供，减少重复样式代码。

表 3-2 前端目录职责

前端目录	职责
app/(auth)	登录、注册、找回密码和第三方登录回调等认证页面。
app/(main)	工作台、探索、知识库、工具、设置、管理后台等主要业务页面。
app/widget	公开 Widget 页面，用于将 Agent 聊天能力嵌入外部页面。
components	业务组件和通用 UI 组件。
lib	HTTP 客户端、API 配置、业务服务和工具函数。

types	前端接口类型定义，降低跨模块传参错误。
hooks	聊天、复制、Agent 表单、RAG 会话等自定义 Hook。

3.4 部署架构设计

部署目录使用 Docker Compose 组织服务。PostgreSQL 使用 pgvector 镜像以支持向量存储，RabbitMQ 提供消息队列和管理端口，后端服务对外暴露 8088 端口并设置 /api 上下文路径，前端服务对外暴露 3000 端口。start.bat 和 start.sh 脚本封装了 local、production、external 和 tools 等启动模式，降低部署复杂度。

表 3-3 Docker Compose 服务组成

服务	镜像或构建方式	作用
postgres	pgvector/pgvector:pg15	保存业务数据和向量数据。
rabbitmq	rabbitmq:3.12-management-alpine	支撑异步消息和任务处理。
backend	deploy/Dockerfile.backend	运行 Spring Boot 后端服务。
frontend	deploy/Dockerfile.frontend	运行 Next.js 前端服务。
adminer	adminer:4-standalone	可选数据库管理工具。

4 核心模块设计与实现

4.1 用户认证与权限模块

用户认证模块是平台入口，主要负责注册、登录、验证码、密码重置、用户资料 and 用户设置等功能。后端通过 LoginController、PortalUserController、UserDomainService 等组件组织认证和用户逻辑，前端在登录、注册和设置页面中调用对应服务。系统还区分普通用户和管理员访问路径，为管理后台提供权限边界。

在安全设计上，系统使用 Token、用户上下文和拦截器控制访问。对于外部接口，系统还提供 API Key 机制，使第三方调用不需要直接使用网页登录态。需要注意的是，在生产环境中，JWT 密钥、邮件密码、GitHub Token 等敏感信息必须通过环境变量或独立密钥配置文件注入，不能保留真实默认值。

4.2 Agent 创建与会话模块

Agent 模块是系统核心。用户可以创建 Agent，配置名称、描述、系统提示词、模型配置、工具和知识库，然后在工作区中发起会话。后端围绕 PortalAgentController、PortalAgentSessionController、AgentMessageHandler、AgentToolManager 等类处理 Agent 的生命周期和消息流程；前端则通过工作台、聊天面板、会话列表和预览组件提供交互。

会话模块需要解决两个问题：一是如何保持用户、Agent、会话和消息之间的关系；二是如何让模型响应以流式方式返回，使用户获得更接近真实聊天的体验。系统在后端提供会话消息接口和消息传输能力，在前端通过聊天组件展示消息列表、输入框、任务状态和中断操作。

图 4-1 Agent 会话处理流程

用户选择 Agent → 创建或进入会话 → 输入问题 → 后端读取 Agent 配置
→ 解析模型、工具和知识库上下文 → 调用 LLM 或工具 → 流式返回消息
→ 保存消息与任务状态 → 前端实时更新对话界面

4.3 RAG 知识库模块

RAG 知识库模块用于解决大模型缺少私有知识的问题。用户可以创建数据集，上传文档，系统对文档进行解析、切分和向量化，再将文档片段存入向量表。用户提问时，系统先根据问题进行相似度检索，再将检索结果作为上下文输入模型，从而生成更贴近用户资料的回答。

从实现上看，RAG 模块涉及多个业务对象：数据集、文件、文档片段、处理进度、发布版本、市场知识库和用户安装关系。后端提供 RagQaDatasetController、FileOperationController、RagSearchController、RagPublishController、RagMarketController 等接口；基础设施层提供 Markdown 处理、文件类型检测、文档树构建、向量化编排和 rerank 能力。

图 4-2 RAG 文件处理与检索流程

文件上传 → 文件类型识别 → 文档解析 → Markdown/文本标准化
→ 结构化切分 → 文档片段入库 → 调用 Embedding 模型生成向量
→ 写入 pgvector → 用户提问时检索相关片段 → 组合上下文生成回答

该模块的难点在于流程较长，并且每一步都可能失败。例如文件格式不兼容、文档内容过长、向量模型配置不完整、数据库向量维度不匹配等问题都会影响最终检索质量。因此系统设计了文件处理状态、进度查询和分层处理逻辑，使用户能够知道文件是否仍在处理中，也便于后端定位具体失败环节。

4.4 工具市场与工具安装模块

工具市场模块使 Agent 不局限于文本生成，而是可以扩展外部能力。用户可以浏览推荐工具、安装工具、查看工具版本，开发者或管理员可以上传、发布和审核工具。系统中存在 market tools、installed tools、user tools 等不同视角，前端也提供工具市场、工具详情、安装对话框和管理后台工具审核页面。

从平台角度看，工具市场相当于 Agent 能力扩展层。它把“工具定义、工具版本、用户安装、市场审核”拆成独立流程，既能保证普通用户使用简单，也能为后续接入更多工具协议或 MCP Gateway 留出扩展空间。该模块的存在使系统更接近智能体平台，而不是单纯的聊天页面。

4.5 管理后台模块

管理后台负责平台治理和运营能力，包括用户管理、Agent 管理、模型供应商管理、工具审核、知识库审核、订单管理、商品管理、规则管理、容器管理和认证配置等。前端在 app/(main)/admin 下组织管理页面，并通过 admin-* service 文件访问后端接口；后端则在 interfaces/api/admin 包下提供相应 Controller。

管理后台的意义在于把系统从个人项目提升为可运营平台。这部分能体现较大的业务完整度，因为平台不仅要让用户使用 AI 功能，还要考虑管理员如何配置模型、审核内容、查看订单和管理资源。这样的设计也为后续多用户、多租户和商业化功能提供基础。

4.6 支付、用量与 API Key 模块

项目中包含商品、订单、支付、账单和用量记录等模块，后端集成了 Stripe 与 Alipay 相关依赖，并在前端提供充值流程、二维码展示、支付方式选择和支付状态轮询

组件。虽然课程作业中不一定需要完整演示真实支付闭环，但这些模块说明系统已经考虑到资源消耗、模型调用成本和平台商业化场景。

API Key 模块用于对外开放 Agent 能力。用户可以创建、查询、重置、禁用或删除 API Key，并将其与 Agent 关联。与网页登录态相比，API Key 更适合程序调用和第三方集成。对于智能体平台而言，对外开放能力是重要扩展方向，也需要配合用量统计和访问限制共同使用。

4.7 容器管理与 Web Terminal 模块

项目还包含容器管理和 Web Terminal 相关能力。后端通过 Docker Java 依赖与 Docker 服务交互，配置中提供容器用户目录、默认 MCP Gateway 镜像和监控周期；前端包含 WebTerminal 组件与容器管理页面。这部分设计可以用于为用户工具或 Agent 运行环境提供隔离容器，增强平台执行外部工具的能力。

5 数据库与数据流设计

5.1 核心数据对象

由于项目功能较多，数据库对象也较丰富。课程报告中不逐一展开所有字段，而是重点说明核心对象及其关系。系统围绕用户、Agent、会话、消息、工具、知识库、文件、文档片段、订单、商品和记忆等对象展开，这些对象共同构成智能体平台的数据基础。

表 5-1 核心数据对象

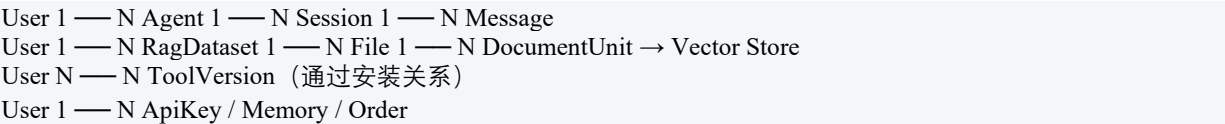
数据对象	含义	主要关系
User	平台用户	可创建 Agent、知识库、订单、API Key 和记忆。
Agent	用户配置的智能体	关联模型配置、工具、知识库和会话。
Session	一次对话会话	属于某个用户和 Agent，包含多条 Message。
Message	会话消息	记录用户输入、模型输出、工具调用结果等内容。

Tool	工具定义或工具版本	可被用户安装，也可进入市场审核流程。
RagDataset	知识库数据集	包含多个文件，可被 Agent 或用户安装使用。
File	知识库文件	经过解析后生成多个 DocumentUnit。
DocumentUnit	文档片段	用于向量化和相似度检索。
Memory	用户记忆	可用于个性化对话和长期上下文管理。
Order/Product	订单与商品	支撑充值、购买和用量管理。
ApiKey	外部访问凭证	用于第三方系统调用 Agent 能力。

5.2 数据关系设计

核心关系可以概括为：一个用户可以创建多个 Agent；一个 Agent 可以拥有多个会话；一个会话包含多条消息；一个用户可以创建多个知识库数据集；一个数据集包含多个文件；一个文件被解析为多个文档片段；文档片段向量化后存入向量表；用户可以安装多个工具，Agent 在会话过程中可根据配置调用工具。

图 5-1 核心实体关系示意



5.3 Agent 会话数据流

会话数据流体现了系统从用户输入到模型输出的完整链路。前端提交消息后，后端首先进行身份校验和会话合法性检查，然后加载 Agent 配置、用户模型配置、工具配置和可用知识库。若启用了 RAG，系统会先检索相关文档片段；若需要工具，则由 Agent 工具管理逻辑进行调用；最后将模型输出通过流式通道返回前端，并保存消息和任务状态。

这种数据流的特点是“请求入口简单、后端编排复杂”。对于用户来说只是输入一个问题，但后端需要同时处理权限、上下文、模型、工具、知识检索和消息持久化。这也是智能体平台与普通 Web CRUD 系统的主要区别之一。

5.4 RAG 文件处理数据流

RAG 文件处理数据流更偏向异步任务。用户上传文件后，系统并不应让请求一直阻塞到全部向量化完成，而是记录文件状态并进入处理流程。处理过程包括文件读取、类型检测、文本提取、结构化切分、片段入库、向量生成和向量存储。前端通过进度接口展示文件处理状态，用户可以在处理完成后进行检索或对话。

该设计避免了大文件处理导致前端长时间等待，也便于在处理失败时给出明确状态。后续如果需要进一步增强，可以将文档解析、向量化和索引更新拆分为更细粒度的消息事件，并引入重试、死信队列和处理日志。

5.5 统一响应与前后端接口

前后端接口采用统一响应结构，通常包含 `code`、`message`、`data` 和 `timestamp` 等字段。前端在 `lib` 中封装 API 请求，并结合 `toast-utils` 对成功或失败消息进行统一提示。这样可以减少每个页面重复处理错误提示的代码，也能让后端错误信息以更一致的方式传递到界面。

6 系统实现与运行部署

6.1 后端实现要点

后端入口类为 `org.xhy.AgentXApplication`，启用了 Spring Boot、定时任务和文件存储能力。`application.yml` 中配置了服务端口 8088 和 `/api` 上下文路径，数据库默认连接 PostgreSQL，RabbitMQ 默认连接本地或容器服务。项目通过 Maven 管理依赖，并使用 Spring Boot Maven Plugin 打包为可运行 Jar。

从业务实现角度看，后端代码数量较多，说明项目不是仅依靠少量接口完成演示，而是围绕用户、Agent、知识库、工具、订单、支付、容器等领域拆分出多个服务。这样的组织方式有助于团队维护，也方便课程报告说明系统工作量。

6.2 前端实现要点

前端使用 Next.js 的 App Router 组织路由，并以组件化方式实现页面。工作台、探索页、知识库、工具市场、管理后台、设置页和 Widget 页面都有对应路由。通用 UI 组件位于 components/ui，业务组件如聊天面板、RAG 聊天对话框、支付流程、工具安装弹窗等则按照功能分类。

前端 API 地址通过 NEXT_PUBLIC_API_BASE_URL 或浏览器当前地址推断。开发环境下默认访问 http://localhost:8088/api，生产环境可使用相对路径 /api，由反向代理转发。这种处理方式使同一套前端代码可以适配本地开发和容器部署。

6.3 部署与启动方式

项目提供 deploy/start.bat 和 deploy/start.sh 两套启动脚本。Windows 环境下可进入 deploy 目录执行 start.bat local，脚本会检查 Docker 与 Docker Compose v2，并在不存在 .env 文件时从模板生成。启动完成后，前端默认访问 http://localhost:3000，后端健康检查地址为 http://localhost:8088/api/health。

表 6-1 部署启动模式

启动模式	用途
local	本地开发或演示，启动 PostgreSQL、RabbitMQ、后端和前端。
production	生产环境模板，仍可通过环境变量调整端口和密钥。
external	使用外部数据库等依赖，仅启动应用相关服务。
tools	在 local 基础上额外启动 Adminer 等辅助工具。

6.4 配置与安全注意事项

在检查项目时发现，配置文件中存在若干默认密钥或外部服务 Token 的配置项。课程报告中不展示具体值，但这类内容在实际提交或部署前应进行清理。更合理的方式是：示例文件只保留变量名和空值，真实密钥放入 application-secrets.yml、部署环境变量或密钥管理系统中，并确保相关文件被 .gitignore 排除。

此外，前端 next.config.mjs 中当前配置了忽略构建期 ESLint 和 TypeScript 错误。这在开发阶段可以减少阻塞，但在生产交付中不建议长期保留。后续应逐步修复类型问题和 lint 问题，再开启构建期校验，以提高系统稳定性。

6.5 线上部署与演示访问

目前系统已经部署到云服务器，可以通过线上地址直接访问系统并验证核心功能。为避免使用个人真实账号，本报告提供专门的测试账号用于访问。

表 6-2 线上演示与源码信息

线上访问地址	https://agent.sakurayuki.me
演示账号	test@qq.com
演示密码	test123
GitHub 源码仓库	https://github.com/314159first/Agent.git

通过线上部署地址，可以依次验证登录、Agent 工作室、知识库、工具市场、记忆管理、执行追踪以及管理后台相关页面。源码仓库用于辅助查看项目结构和主要实现。

7 测试与验证

7.1 测试思路

对于课程作业而言，测试部分不应只写“系统运行正常”，而应从功能、接口、部署和异常场景四个维度说明验证方式。由于本项目是综合性平台，测试重点应放在核心链

路上，包括用户登录、Agent 创建、会话问答、知识库上传、工具安装、管理后台审核和 Docker 启动。

7.2 功能测试用例

表 7-1 功能测试用例设计

测试项	测试方法	预期结果	结果说明
用户登录	输入正确账号密码并提交	返回 Token 并进入主页面	验证认证入口可用。
创建 Agent	填写 Agent 名称、描述和提示词	Agent 创建成功并出现在工作台	验证 Agent 基础管理流程。
发起会话	在 Agent 工作台输入问题	前端展示用户消息和模型回复	验证核心问答链路。
知识库上传	创建数据集并上传文档	文件进入处理流程并可查询进度	验证 RAG 数据入口。
RAG 检索问答	基于上传文档提问	回答能够引用或体现文档内容	验证知识增强效果。
工具安装	在工具市场选择工具并安装	工具进入用户已安装列表	验证工具扩展能力。
管理员审核	管理员进入后台查看工具或知识库	可查看详情并执行审核操作	验证平台治理功能。
支付状态	发起充值或订单流程	订单状态可轮询更新	验证订单流程设计。
健康检查	访问 /api/health	返回成功响应	验证后端服务可用。

7.3 接口与部署测试

接口测试主要关注统一响应格式、权限拦截和异常提示。例如未登录访问需要登录的接口时应返回明确错误；普通用户访问管理员接口时应被拒绝；文件处理失败时应能查询到失败状态而不是前端无响应。部署测试则关注 Docker Compose 是否能正常构建镜像、启动容器、通过健康检查，以及前端能否正确访问后端 API。

表 7-2 接口与部署测试关注点

验证类别	验证内容	关注点
接口格式	检查 code、message、data、timestamp 是否一致	便于前端统一处理。

权限控制	未登录、普通用户、管理员分别访问不同接口	避免越权访问。
异常处理	模拟文件解析失败、模型配置为空、数据库不可用	错误信息应清晰。
部署启动	执行 Docker Compose 启动脚本	容器状态、端口和健康检查。
前后端联调	前端页面调用后端接口	API 地址、跨域和 Token 传递。
数据持久化	创建 Agent、会话和知识库后重启服务	核心数据应保留。

7.4 测试结论

通过上述测试设计，可以覆盖本项目的主要业务链路。当前项目中可见的自动化测试文件较少，更多测试仍依赖手工验证和接口联调。作为后续改进，应逐步补充后端单元测试、应用服务集成测试、前端组件测试和端到端测试，尤其是 Agent 会话、RAG 文件处理和权限控制这三类高风险模块。

8 项目难点与解决方案

8.1 多业务域带来的复杂度

本项目同时包含 Agent、RAG、工具市场、用户、支付、管理后台和容器管理等多个业务域。如果把所有逻辑直接写在 Controller 或前端页面中，代码会快速变得难以维护。项目通过后端分层和前端服务封装降低复杂度，使不同业务域可以相对独立地演进。

8.2 RAG 处理链路较长

RAG 模块并不是简单地把文档传给模型，而是需要经历上传、解析、切分、向量化、入库、检索和回答生成等步骤。每一步都有失败可能，也会影响最终问答质量。项目通过文件状态、处理进度、文档片段和向量存储等对象拆解流程，使用户和系统都能追踪处理进展。

8.3 Agent 会话需要兼顾实时性和持久化

用户希望模型回答能够实时显示，但系统又需要保存会话、消息、任务和工具调用状态。为兼顾这两点，后端需要提供流式消息传输能力，前端需要能够实时渲染消息，同时在流程结束后保持数据一致。这比普通请求响应式接口更复杂，也是智能体应用的核心工程难点之一。

8.4 部署依赖较多

系统运行依赖 PostgreSQL、pgvector、RabbitMQ、后端、前端以及可能的对象存储和 Docker 环境。如果手工配置，课程演示和后续部署都容易出错。项目通过 Docker Compose 和启动脚本将依赖集中管理，减少环境差异造成的问题。

8.5 安全与配置管理

AI 平台通常需要接入模型 API Key、邮件密码、支付密钥、GitHub Token 等敏感信息。项目已经使用环境变量形式配置许多参数，但部分默认值仍需要清理。解决方案是将所有真实密钥移出源码，只保留 .env.example 或 application-secrets.yml.example，并在部署时通过环境变量注入。

表 8-1 项目难点与解决方案总结

问题	解决方式	改进效果
业务模块多	后端分层、前端按页面/组件/服务拆分	降低代码耦合，便于维护。
RAG 流程长	拆分文件状态、文档片段、向量化和进度查询	提高可追踪性和稳定性。
会话需要实时返回	使用流式消息传输和前端实时渲染	改善用户体验。
部署依赖复杂	使用 Docker Compose 统一编排	降低环境搭建成本。
敏感配置风险	改为环境变量和密钥文件	减少泄露风险。
测试覆盖不足	补充单元、集成和端到端测试	提高交付质量。

9 总结与展望

9.1 工作总结

本文围绕 AgentX 智能体平台进行了课程作业报告撰写。项目实现了一个较完整的 AI Agent 平台，涵盖用户认证、Agent 创建、会话交互、模型配置、RAG 知识库、工具市场、记忆管理、支付订单、API Key、管理后台和 Docker 部署等功能。系统采用 Spring Boot 与 Next.js 的前后端分离架构，并通过 PostgreSQL、pgvector、RabbitMQ 和 Docker Compose 支撑数据存储、向量检索、异步任务和部署运行。

从工程角度看，项目的主要价值在于将大模型应用所需的多个能力整合为一个平台，而不是只完成单一问答功能。后端分层设计增强了业务可维护性，前端组件化和服务封装提升了页面开发效率，部署脚本和容器编排降低了运行门槛。对于研究生课程作业来说，该项目能够体现一定的系统设计能力、工程实现能力和问题分析能力。

9.2 不足之处

项目目前仍存在一些不足。第一，自动化测试覆盖不足，许多核心链路仍需要依赖手工验证；第二，部分配置项存在默认敏感值，生产环境必须清理；第三，前端构建阶段暂时忽略了 TypeScript 和 ESLint 错误，不利于长期维护；第四，RAG 检索效果仍可通过更精细的分块策略、rerank 和召回评估进一步优化；第五，Agent 工具调用链路的可观测性还可以增强，例如增加调用耗时、输入输出摘要和失败原因记录。

9.3 后续展望

后续可以从四个方向继续完善：其一，补充后端单元测试、接口集成测试和前端端到端测试；其二，加强安全配置管理，移除源码中的真实密钥并完善权限模型；其三，优化 RAG 质量，包括文档清洗、结构化切分、混合检索和 rerank；其四，增强 Agent 运行观测能力，使工具调用、模型调用、知识检索和任务执行过程更透明。若进一步发展，该项目可以扩展为多租户智能体平台，支持更多模型供应商、工具协议和企业级知识库场景。

附录 A 主要代码说明

表 A-1 主要代码位置

代码位置	说明
AgentX/src/main/java/org/xhy/AgentXApplication.java	后端应用入口，启用 Spring Boot、调度和文件存储。
AgentX/src/main/java/org/xhy/interfaces/api	后端接口层，包含 portal、admin、public_api、external 等接口。
AgentX/src/main/java/org/xhy/application	应用服务层，组织 Agent、RAG、订单、任务等业务用例。
AgentX/src/main/java/org/xhy/domain	领域层，包含用户、会话、容器、订单等领域服务与仓储。
AgentX/src/main/java/org/xhy/infrastructure	基础设施层，适配 Docker、LLM、支付、消息队列、RAG 解析等能力。
agentx-frontend-plus/app/(main)	前端主应用页面，包括工作台、知识库、工具、设置和管理后台。
agentx-frontend-plus/components	前端业务组件和通用 UI 组件。
agentx-frontend-plus/lib/api-config.ts	前端 API 地址与接口路径配置。
deploy/docker-compose.yml	服务编排文件，定义数据库、消息队列、后端、前端和工具服务。

附录 B 系统运行效果截图

本附录对上传的系统截图进行了筛选和整理，保留能够体现项目核心功能、平台化能力和管理端能力的代表性页面。截图来源于已部署的线上系统，访问地址为 <https://agent.sakurayuki.me>。

B.1 用户端核心功能截图



图 B-1 登录与身份认证页面

该页面展示了系统的用户入口，支持账号密码登录和 GitHub 登录，体现了平台在用户认证与第三方登录方面的基础能力。

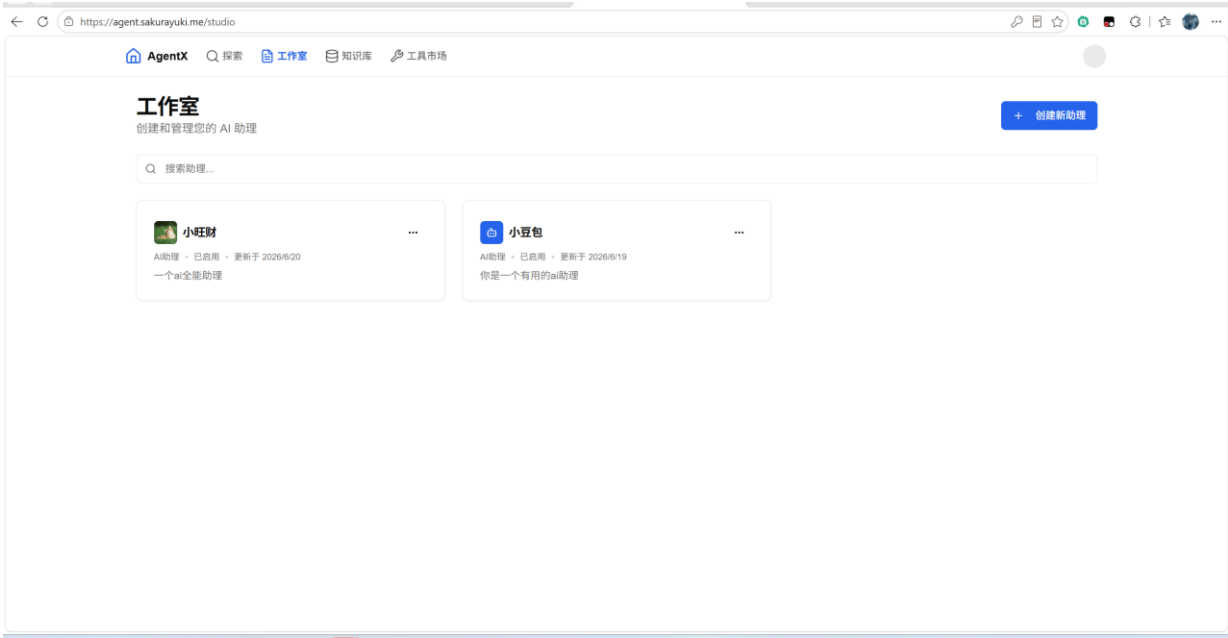


图 B-2 Agent 工作室页面

工作室页面集中展示用户创建的 AI 助手，支持创建、搜索和管理，是用户使用 Agent 能力的主要入口。

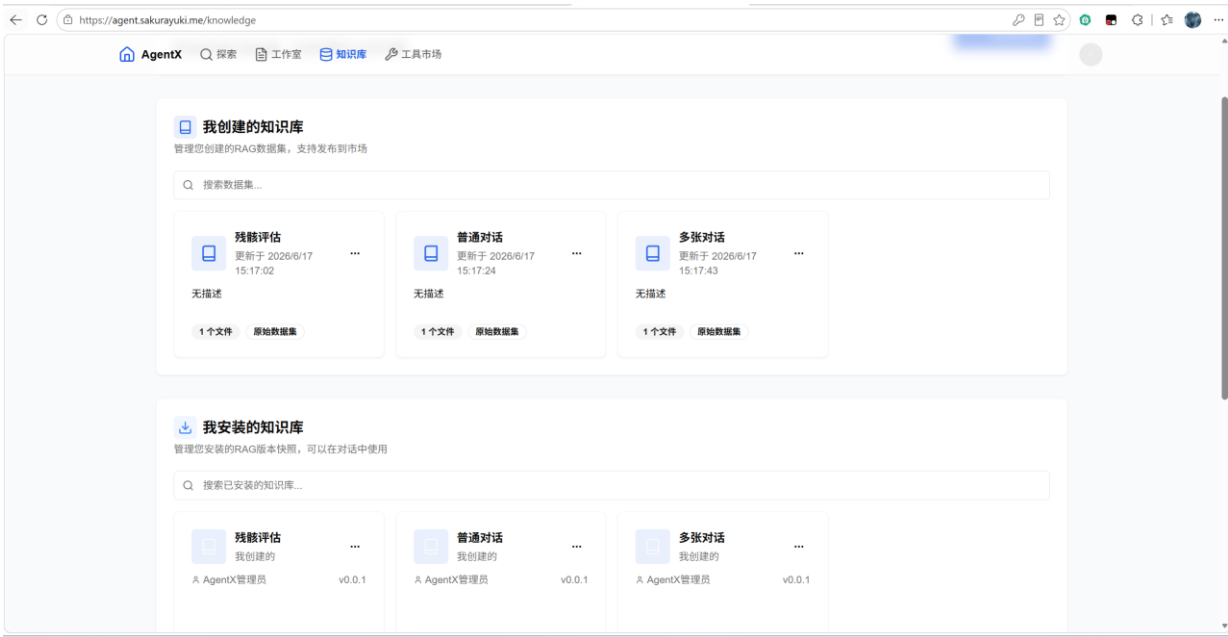


图 B-3 知识库管理页面

知识库页面展示了用户创建和安装的 RAG 数据集，能够体现系统对私有知识库、版本发布和数据集管理的支持。

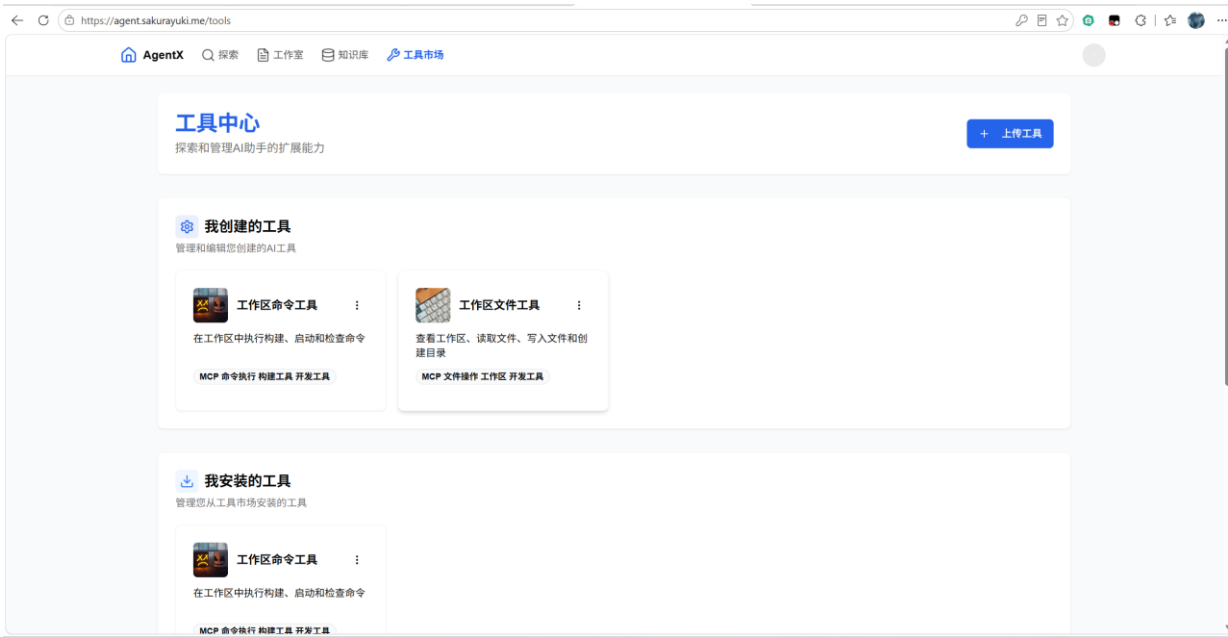


图 B-4 工具中心页面

工具中心展示用户创建和安装的 MCP 工具，说明平台不仅支持模型问答，也支持通过工具扩展 Agent 的执行能力。



图 B-5 默认模型与 RAG 模型配置页面

通用设置页面展示默认对话模型、OCR 模型和嵌入模型配置，说明系统对多模型能力进行了统一管理。

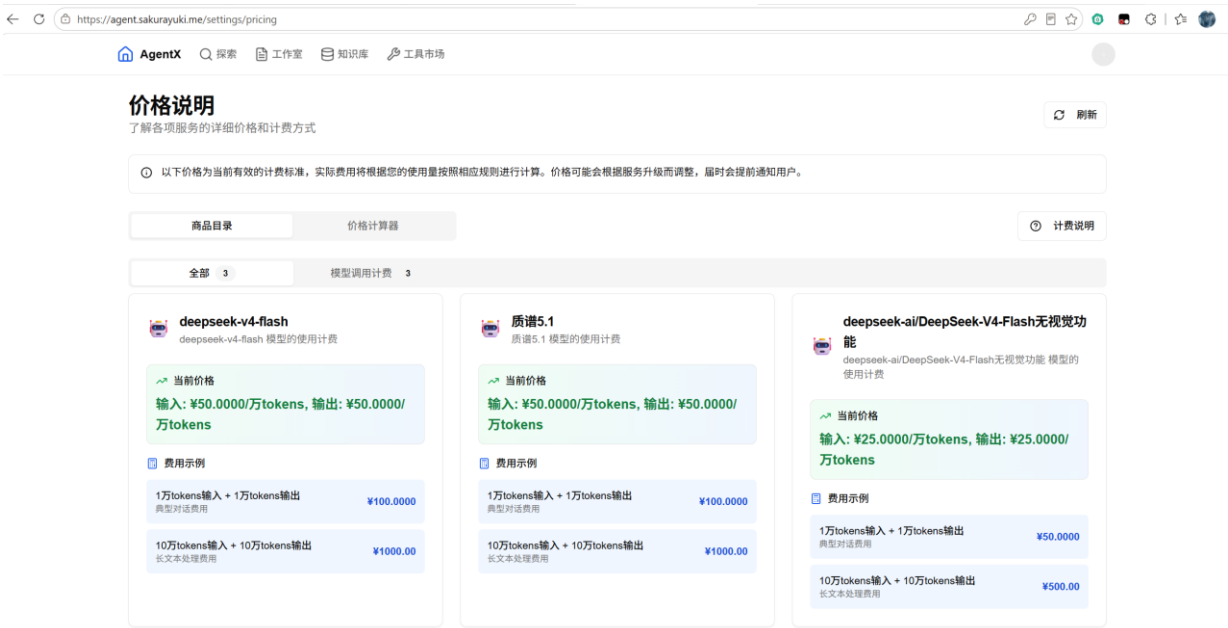


图 B-6 价格说明页面

价格说明页面展示模型调用计费规则，体现系统对模型调用成本、商品配置和用量计费的考虑。



图 B-7 记忆管理页面

记忆管理页面展示用户记忆的查看和维护能力，是实现长期上下文和个性化交互的重要组成部分。

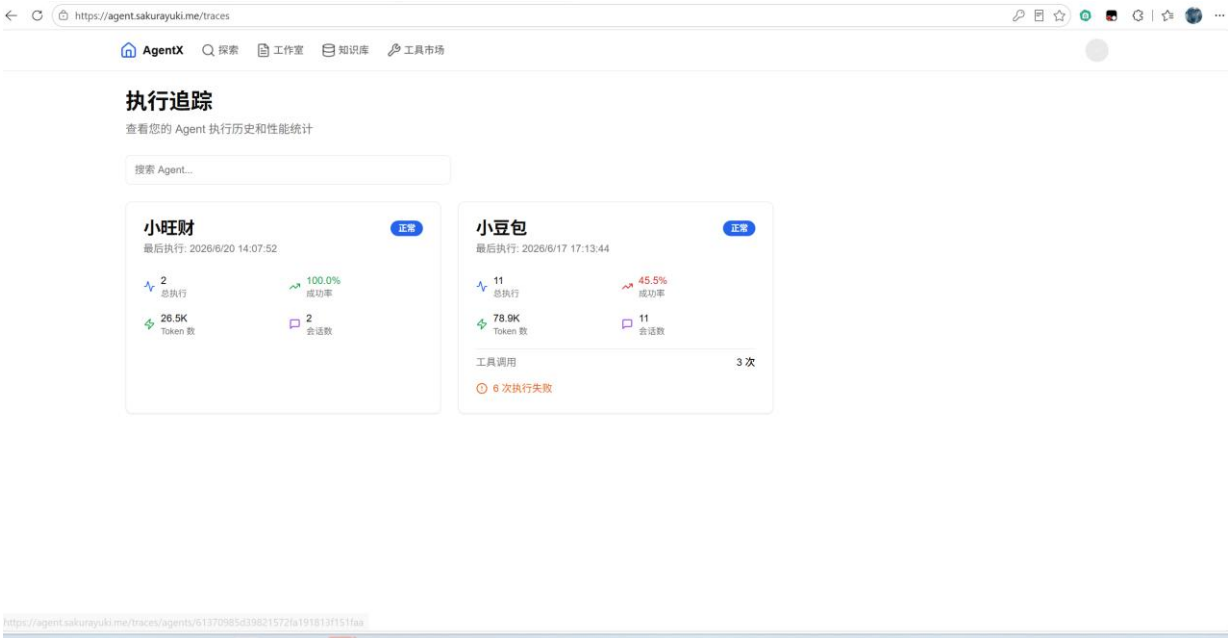


图 B-8 Agent 执行追踪页面

执行追踪页面展示 Agent 的执行次数、成功率、Token 数和会话数，有助于分析 Agent 运行效果和性能状态。

B.2 管理后台功能截图

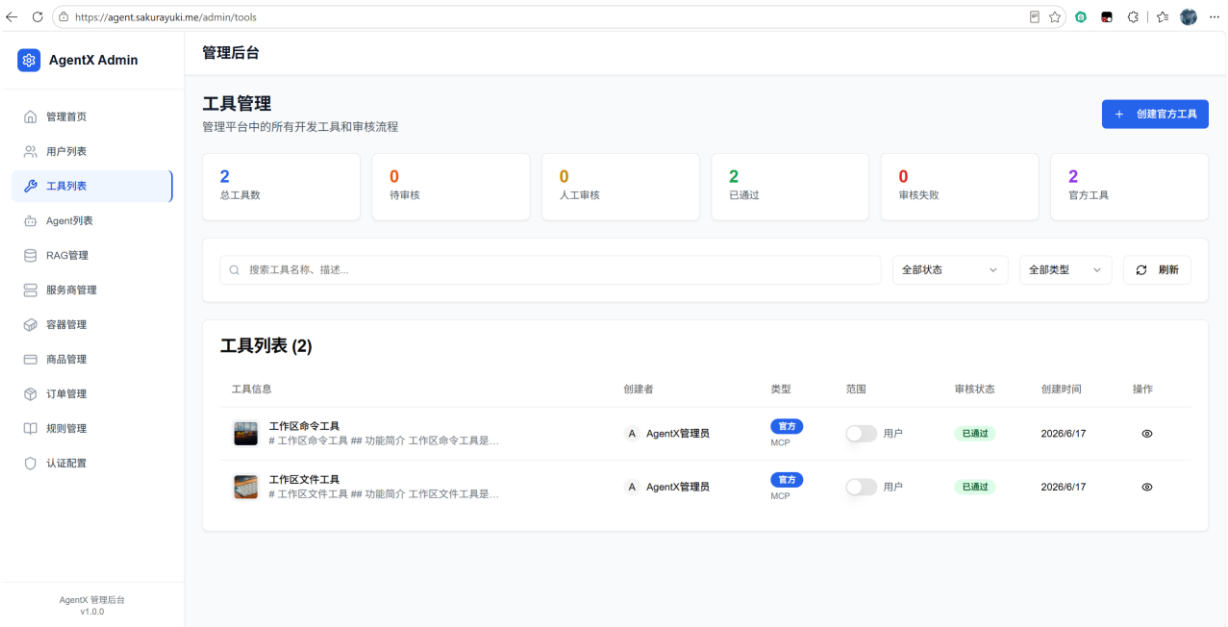


图 B-9 管理后台工具管理页面

工具管理页面展示工具数量、审核状态和工具列表，体现管理员对平台工具生态的治理能力。

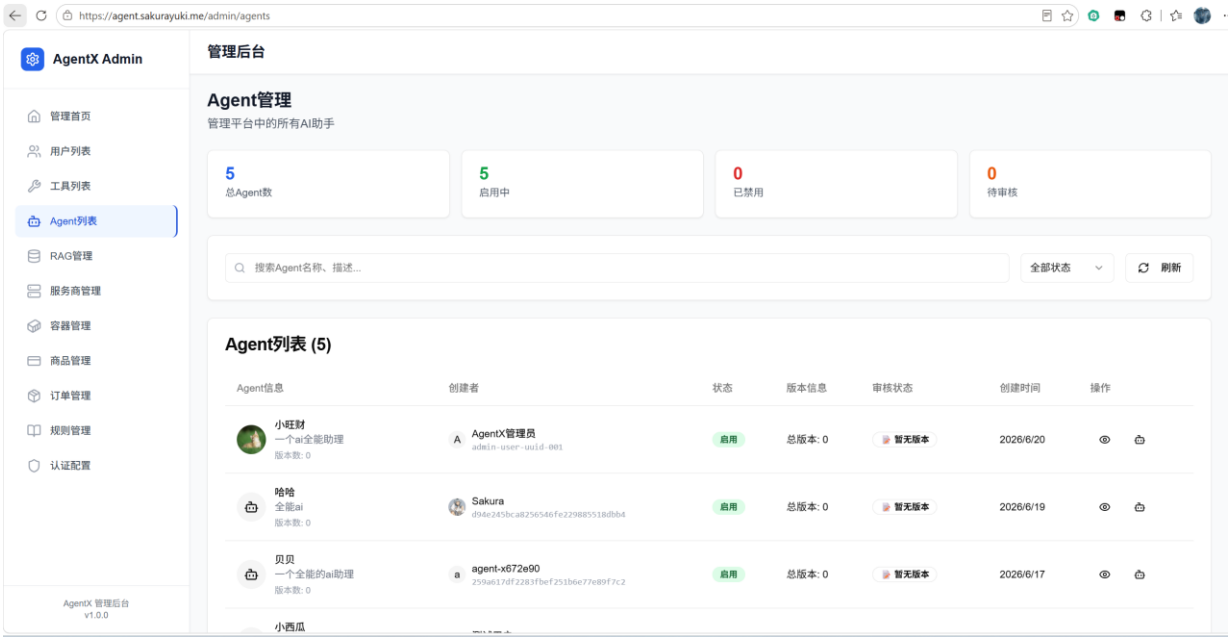


图 B-10 管理后台 Agent 管理页面

Agent 管理页面展示平台内全部 Agent 的状态、创建者、版本和审核信息，便于管理员进行统一运维。

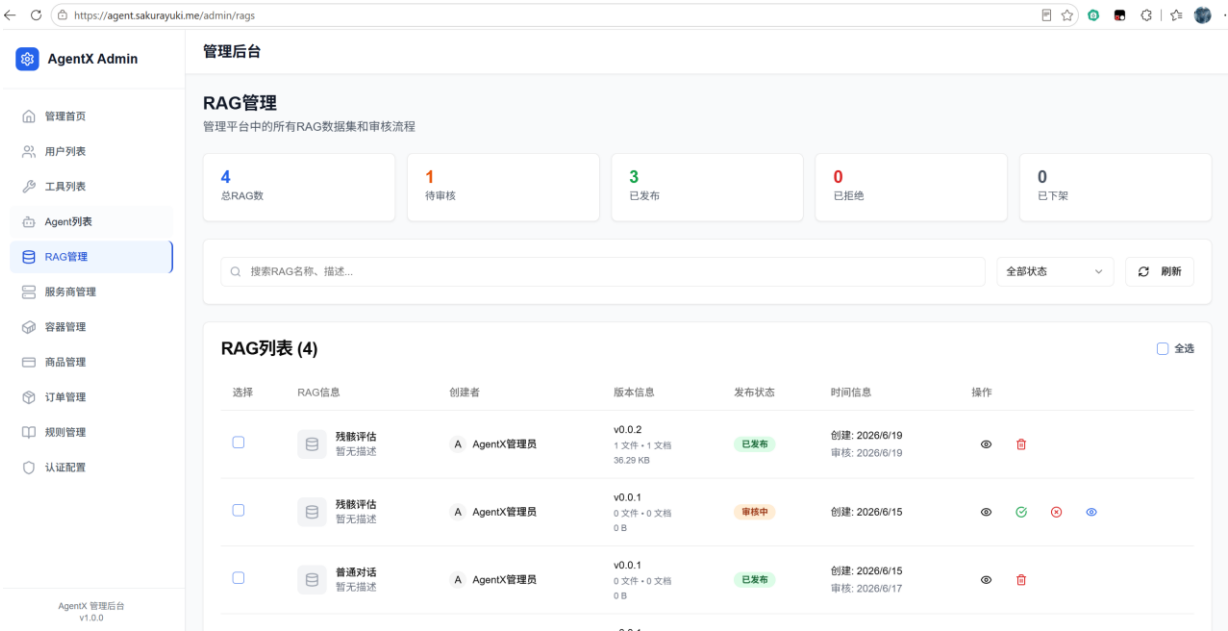


图 B-11 管理后台 RAG 管理页面

RAG 管理页面展示知识库发布、审核和上下架状态，说明系统支持知识库市场化和内容审核流程。

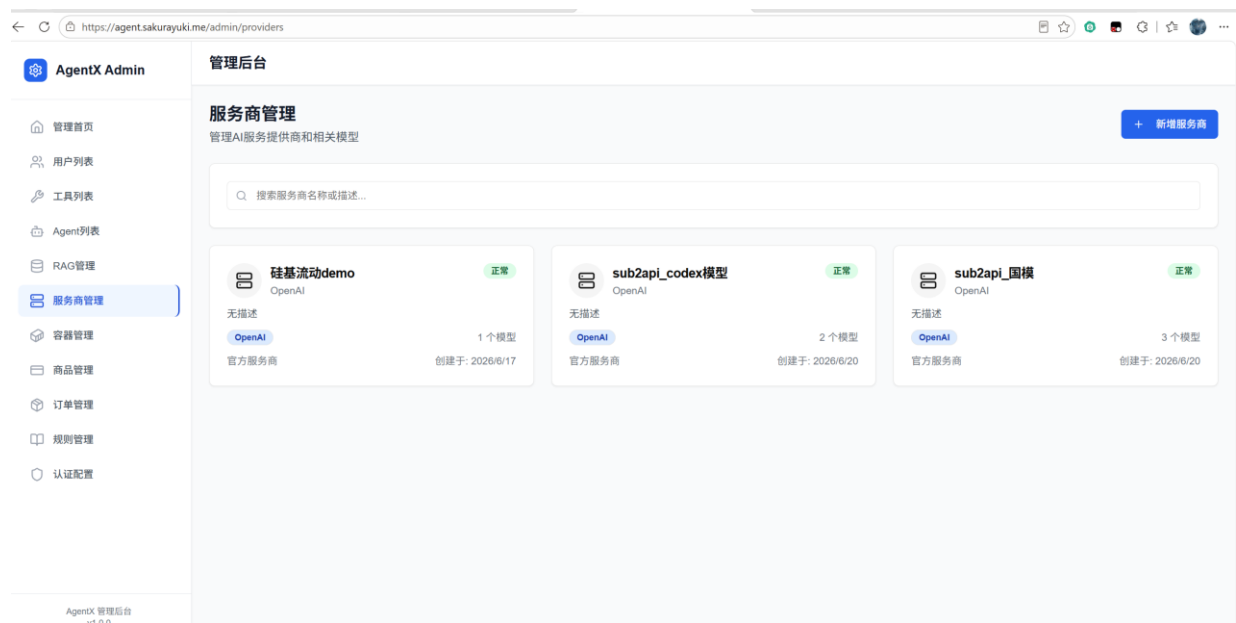


图 B-12 管理后台服务商管理页面

服务商管理页面展示不同模型服务商及其状态，为多模型供应商接入和统一管理提供支撑。

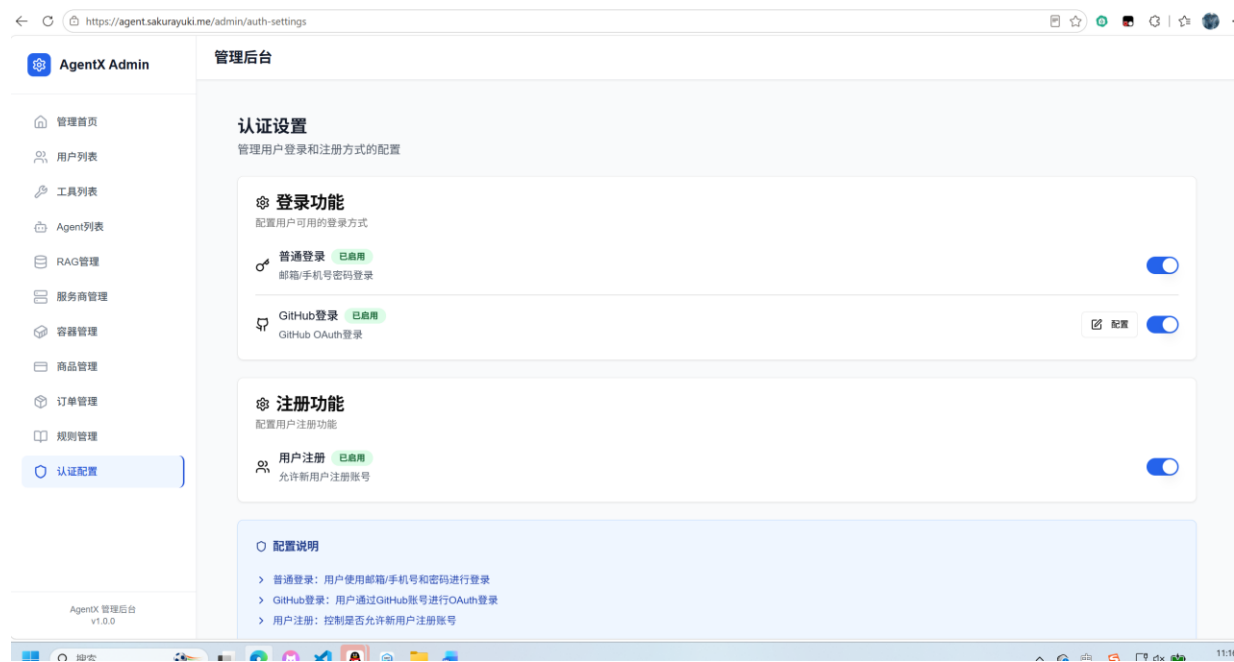


图 B-13 管理后台认证配置页面

认证配置页面展示普通登录、GitHub 登录和用户注册开关，体现平台认证方式可配置的设计。